
SecureBuild

Multi-Gate CI/CD Security Pipeline

Secure Software Design & Development

Submitted By

Name	Reg. No.
Muhammad Adeel Haider	241541
Wazeema Eeman	241493

Submitted To

Ms. Maryam Mehmood

*Lecturer, Secure Software Design &
Development*

Department

National Cyber Security Academy (NCSA)
Air University, Islamabad

June 1, 2026

1 Introduction

One of the most common problems in modern software development is that security tends to get treated as something you deal with at the end of the project, or even after something breaks in production. This is obviously not ideal. The whole idea behind shifting security left is to catch issues as early as possible, ideally before the code is even merged into the main branch.

The project I built, **SecureBuild**, is a security gate tool that plugs directly into a CI/CD pipeline. Every time a developer pushes code, SecureBuild runs five security checks in parallel against the entire codebase. If it finds anything critical, the pipeline fails and the deployment is blocked. If everything is fine, the pipeline passes and the code moves forward.

The tool covers five separate threat categories that I think together give a pretty comprehensive picture of a codebase's security posture:

1. **Hardcoded secrets and credentials** — things like API keys, AWS access keys, passwords, and private keys sitting in source files where they should never be.
2. **Code-level vulnerabilities** — SQL injection, command injection, insecure deserialization, weak crypto, and similar dangerous patterns that static analysis can catch.
3. **Vulnerable dependencies** — third-party packages with known CVEs, or packages that have not been updated in two years and may be accumulating unpatched vulnerabilities.
4. **Licence compliance** — dependencies whose licences conflict with how the software is being distributed, which is a legal risk that often gets ignored until it becomes expensive.
5. **Infrastructure misconfiguration** — Docker containers running as root, Kubernetes pods without resource limits, GitHub Actions workflows that are vulnerable to script injection, and similar IaC security issues.

Beyond the scanning itself, the project also includes a weighted risk scoring engine, a remediation engine with 44 fix templates, a Flask web dashboard, a full REST API with Bearer token authentication, HTML/PDF/JSON report generation, and integrations with GitHub, GitLab, Slack, and email. I will go through all of this in detail throughout the report.

2 System Architecture

2.1 Overall Design

The system is built around a central **Orchestrator** that coordinates everything. When a scan is triggered, the orchestrator validates the repository path, collects git metadata (branch name, commit hash), then hands off to the Gate Runner which executes all five security gates in parallel using Python's `ThreadPoolExecutor`. Once the gates finish, results go to the Risk Scorer, then to the database for persistence, then to the report generator, and finally to any configured notification channels.

The design was intentionally kept modular. Each gate is completely independent from every other gate, which means a bug or error in one gate cannot affect the results of any other. It also makes it easy to add a new gate later without touching any existing code.

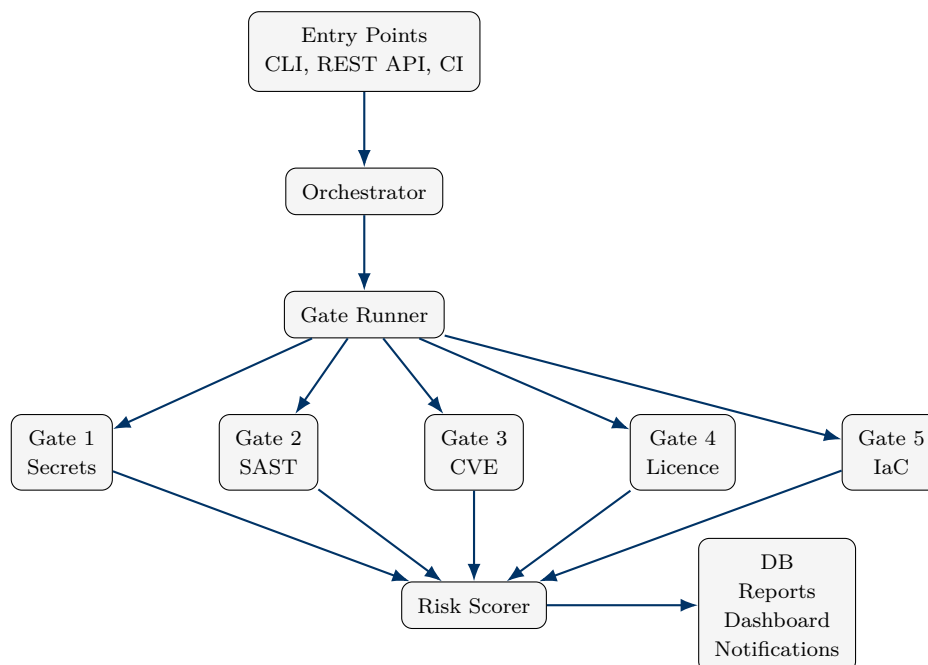


Figure 1: High-level scan pipeline

2.2 Scan Lifecycle



Figure 2: Scan lifecycle at a glance

Each phase produces a structured `RunResult` that is persisted to SQLite and reused for reports, dashboards, and notifications.

2.3 Module Breakdown

The project is organized into a small set of core layers with clear ownership.

Module	Responsibility
<code>cli.py</code>	CLI entry point, command routing, exit codes, and local report viewing.
<code>engine/</code>	Orchestrator, gate runner, config validation, models, logging, and utilities.
<code>gates/</code>	Five independent security gates plus shared helpers (entropy, CWE mapping, IaC discovery).
<code>scoring/</code>	Weighted risk scoring, trend analysis, and remediation matching.
<code>reporter/</code>	HTML/JSON report generation and optional PDF export.
<code>dashboard/</code>	Flask UI, run views, and REST API endpoints.
<code>integrations/</code>	GitHub/GitLab clients, Slack/email notifications, SVG badge generation.
<code>data/</code>	Remediation templates and bundled security knowledge.

Table 1: Key modules and responsibilities

3 Security Gates

3.1 Gate Coverage Summary

Gate	Focus	Representative checks
Gate 1: Secrets	Hardcoded secrets in code and env files	AWS keys, GitHub PATs, JWTs, PEM keys, high-entropy strings, .env credentials.
Gate 2: SAST	Code-level vulnerability patterns	SQL/command injection, unsafe deserialization, weak crypto, insecure subprocess usage.
Gate 3: CVE	Dependency risk	Known vulnerable versions, affected ranges, stale packages.
Gate 4: Licence	Licence compliance	SPDX normalisation, project-type risk classification, allow/block overrides.
Gate 5: IaC	Infrastructure misconfiguration	Docker, Compose, Kubernetes, and GitHub Actions checks.

Table 2: Gate coverage summary

3.2 Detection Highlights

Gate 1: Secrets. Pattern matching and Shannon entropy scanning operate together to catch obvious and obfuscated credentials. Findings are masked before storage, and inline `# nsec` or `// nsec` suppressions are respected.

Gate 2: SAST. The gate prefers Bandit or Semgrep when available and falls back to a built-in AST scanner, so it always produces baseline SAST coverage.

Gate 3: CVE Audit. Dependencies are parsed from `requirements.txt` and `package.json`, then matched against a bundled CVE database with version-range checks and stale-package detection.

Gate 4: Licence. SPDX normalisation and project-type rules classify licences by risk, with configurable allow/block overrides.

Gate 5: IaC. Dockerfiles, Compose files, Kubernetes manifests, and GitHub Actions workflows are scanned for high-impact misconfigurations.

Test fixtures. The vulnerable and fixed fixtures under `tests/fixtures/` provide positive and negative coverage for each gate.

4 Risk Scoring Engine

4.1 The Scoring Formula

The risk score aggregates all findings from all five gates into a single number between 0 and 100. A higher number means higher risk. The formula is:

$$\text{contribution}_i = \text{cvss}_i \times w_{g_i} \times m_{s_i}$$

$$\text{raw score} = \frac{\sum_{i=1}^n \text{contribution}_i}{\sum_{i=1}^n (w_{g_i} \times m_{s_i})}$$

$$\text{overall} = \min(\text{raw score}, 10) \times 10$$

where w_{g_i} is the gate weight for the gate that produced finding i , and m_{s_i} is the severity multiplier for the finding's severity level.

4.2 Weights and Multipliers

Gate	Weight	Severity	Multiplier
Secrets	1.5	Critical	2.0
SAST	1.3	High	1.5
CVE	1.2	Medium	1.0
IaC	1.1	Low	0.5
Licence	0.8	Info	0.2

The weights reflect how immediately exploitable a finding from each gate is. Secrets get the highest weight because a leaked AWS key can be used within minutes of discovery. Licence issues get the lowest weight because they are a legal risk, not an immediate technical one. The critical severity multiplier of 2.0 means a critical finding contributes twice as much to the score as the same CVSS score at medium severity, which ensures critical issues are never buried by a pile of low-severity noise.

4.3 Risk Levels and Blocking Rules

Score Range	Risk Level	Pipeline Action
80–100	Critical	Deployment blocked immediately
60–79	High	Block; address critical/high findings
40–59	Medium	Block if thresholds exceeded; prioritise fixes
20–39	Low	Pass; address in regular maintenance
0–19	Minimal	Pass; continue with standard practices

Table 3: Risk thresholds and pipeline actions

The pipeline is blocked if any of these conditions is true: (1) any critical finding exists, (2) five or more high findings exist across all gates, or (3) the overall risk score exceeds 7.0 on the 0–10 internal scale. All three thresholds are configurable via `securebuild.yaml`. Setting `warn_only: true` disables blocking entirely and makes the tool report-only.

4.4 Trend Analysis and Explainability

The scorer compares the last five runs for the same repository and labels the trend as `improving`, `stable`, `degrading`, or `critical_regression`. It also supports `simulate_fix()` to estimate score changes and generates a short narrative of the top contributing findings. A percentile rank across historical runs provides additional context.

5 Remediation Engine

The remediation engine lives in `scoring/remediation.py` and loads 44 fix templates from `data/remediation_templates.yaml`. Each template is keyed by one of: a combined

gate+CWE identifier (e.g., `sast_CWE-89`), a gate+rule identifier, a CWE alone, or a finding type alone. The matching priority goes from most specific to least specific. Each template includes a short explanation, before/after code snippets, an effort estimate, and reference links. The engine also flags **quick wins** — low-effort findings with high security impact — so developers can prioritise fixes efficiently.

6 Database Design

6.1 Schema

The database uses SQLite in WAL (Write-Ahead Logging) mode for better read concurrency. Tables are auto-created from `schema.sql` on first run.

Table	Purpose
<code>pipeline_runs</code>	One row per scan execution with repo metadata, score, status, and duration.
<code>gate_results</code>	Per-gate summaries including severity counts and timing.
<code>findings</code>	Full finding detail with file, line, CWE/rule IDs, and fix diff.
<code>api_keys</code>	Hashed API keys with audit fields and revocation flag.

Table 4: SQLite schema overview

6.2 Security Properties of the Database Layer

All SQL statements are parameterised via the Python `sqlite3` placeholder syntax to prevent injection from scan data. API keys are generated with `secrets.token_hex(32)` and stored only as SHA-256 hashes, with a revocation flag retained for auditability.

7 Web Dashboard and REST API

7.1 Flask Dashboard

The dashboard uses Flask Blueprints to keep the home view, run history, and repository summaries cleanly separated. The run detail view surfaces file and line context, severity badges, CVSS, CWE identifiers, and remediation diffs.

7.2 REST API

The REST API exposes the same data as the dashboard and requires a **Authorization: Bearer <key>** header for all endpoints except health. Missing headers return HTTP 401; invalid keys return HTTP 403.

Method	Endpoint	Description
GET	/api/v1/health	Health check (no auth)
POST	/api/v1/scan	Trigger a new scan
GET	/api/v1/runs	Paginated run list
GET	/api/v1/runs/{id}	Full run detail
GET	/api/v1/runs/{id}/stream	SSE real-time progress
GET	/api/v1/repos/{name}	Repo aggregate view
GET	/api/v1/findings	Filtered findings query

Table 5: Key REST API endpoints

The `/api/v1/runs/{id}/stream` endpoint uses **Server-Sent Events** to push real-time scan progress without polling.

8 Reporting

Four output formats are generated after every scan.

Format	Purpose
HTML	Self-contained report with summary cards, charts, and full findings.
PDF	Printable audit-ready export (skipped if WeasyPrint is unavailable).
JSON	Complete <code>RunResult</code> for integration and automation.
Markdown	<code>summary.md</code> for GitHub Actions job summaries.

Table 6: Report outputs

9 CI/CD Integration and Containerisation

9.1 GitHub Actions

The bundled workflow at `.github/workflows/securebuild.yml` demonstrates minimal permissions, full git history, and report artifacts with a job summary. If critical issues are found, the pipeline exits with code 1 to block the PR.

9.2 Docker Containerisation

The `Dockerfile` uses a multi-stage build and runs as a non-root `securebuild` user with a `HEALTHCHECK` on the dashboard API. `docker-compose.yml` starts the scanner and dashboard with a shared data volume.

9.3 Pre-commit Hooks

The `.pre-commit-config.yaml` runs Black, Flake8, isort, and `detect-secrets` with a baseline for known false positives.

10 External Integrations

SecureBuild integrates with common developer workflows to surface findings where teams already work.

Integration	Capabilities
GitHub	PR summary and inline comments, commit status updates, job summary.
GitLab	MR comments and pipeline status updates.
Slack	Block Kit notifications with top findings and score.
Email	HTML executive summaries via SMTP.
Badges	SVG score and per-gate status badges for README files.

Table 7: Integration channels

11 Testing

The test suite uses pytest and covers all major components, with fixtures for both vulnerable and fixed sample repositories.

Area	Coverage
Gates	Detection logic, entropy checks, and per-gate findings.
Orchestrator	End-to-end pipeline lifecycle and error isolation.
Scoring	CVSS weighting, trends, and score explanations.
Database	CRUD operations, API key hashing, and query filters.
API	Auth, pagination, SSE stream, and error handling.

Table 8: Test coverage focus

Fixtures under `tests/fixtures/` provide positive and negative regression coverage for each gate and the overall pipeline.

12 Security Principles in the Tool Itself

Beyond scanning other people’s code, the tool itself follows secure design principles:

- **Fail-safe defaults:** critical findings block the pipeline, and gate exceptions are surfaced rather than ignored.

- **Least privilege:** the container runs non-root and CI permissions are minimized.
- **Credential hygiene:** secret values are masked, and API keys are stored as SHA-256 hashes only.
- **Defence in depth:** dual detection in Gate 1 and multi-tool fallback in Gate 2 reduce single points of failure.
- **Auditability:** structured logging and a custom exception hierarchy provide traceable context.

13 Configuration System

SecureBuild is configured via `securebuild.yaml`, and `python cli.py init` generates a starter file with sane defaults.

Section	Purpose
Project	Name, type (open_source/commercial), and language selection.
Gates	Per-gate enablement and settings (patterns, incremental scan, IaC toggles).
Thresholds	Per-severity limits and block/warn behavior.
Exclusions	Global ignore patterns for files and directories.
Notifications	Slack and SMTP settings with per-channel enable flags.
Integrations	GitHub comment and status updates.
Reporting	Output formats, chart inclusion, and per-gate limits.

Table 9: Key configuration sections

14 Conclusion

SecureBuild is a complete, end-to-end CI/CD security gate tool. Starting from the design principle of keeping security checks as early as possible in the development process, I built a system that runs five independent parallel security gates, produces a quantitative risk score, provides actionable remediation guidance with 44 fix templates, persists all results for trend analysis, and integrates with the major platforms developers actually use (GitHub, GitLab, Slack, email).

The modular gate architecture, weighted scoring, and multi-tool SAST fallback produce actionable results without sacrificing reliability. The roadmap for a v2 is clear: keep CVE data fresh and add git history scanning for secrets.

A Repository Snapshot

Path	Contents
engine/	Orchestrator, runner, config, models, logging, database.
gates/	Five security gates plus entropy, CWE mapping, IaC discovery.
scoring/	Risk scoring and remediation engine.
reporter/	HTML/JSON reporting and PDF export.
dashboard/	Flask UI and REST API routes.
integrations/	GitHub/GitLab clients, notifications, badges.
data/	Remediation templates.
docs/	Architecture, configuration, deployment, testing guides.
tests/	pytest suite and fixtures.

Table 10: Repository snapshot (condensed)

B Worked Risk Score Example

Finding	Gate	CVSS	w_g	Sev.	m_s	Contrib.
AWS Access Key	secrets	9.1	1.5	Critical	2.0	27.30
SQL Injection	sast	9.8	1.3	Critical	2.0	25.48
Weak Hash (MD5)	sast	5.3	1.3	Medium	1.0	6.89
Stale Dep.	cve	7.5	1.2	High	1.5	13.50
Missing Limits	iac	5.5	1.1	Medium	1.0	6.05
Sum of contributions						79.22
Sum of weights ($1.5 \times 2.0 + 1.3 \times 2.0 + 1.3 \times 1.0 + 1.2 \times 1.5 + 1.1 \times 1.0$)						9.80
Raw score = $79.22 \div 9.80$						8.08
Overall score = 8.08×10						80.8

Table 11: Result: risk level Critical, pipeline blocked